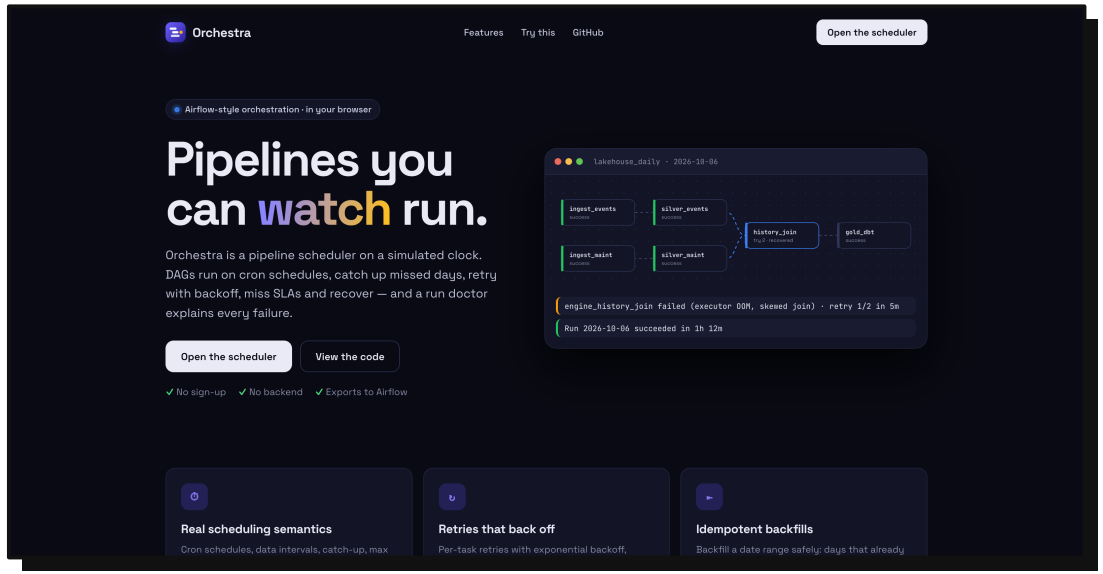
01

ORCHESTRA

Pipelines you can watch run.

An Airflow-style scheduler on a simulated clock: cron, catch-up, retries with backoff, idempotent backfills, SLAs, and a doctor that explains every failure.



TYPESCRIPT / REACT / YAML DAGS / AIRFLOW 2 EXPORT / VITEST / GITHUB PAGES

02

THE PROBLEM: ORCHESTRATION IS INVISIBLE

Scheduling is subtle

Data intervals, logical dates and catch-up confuse even experienced engineers. A run for 1 Oct starts on 2 Oct.

Failures hide in retries

A flaky Spark join passes on its second try. Dashboards stay green while the SLA quietly slips.

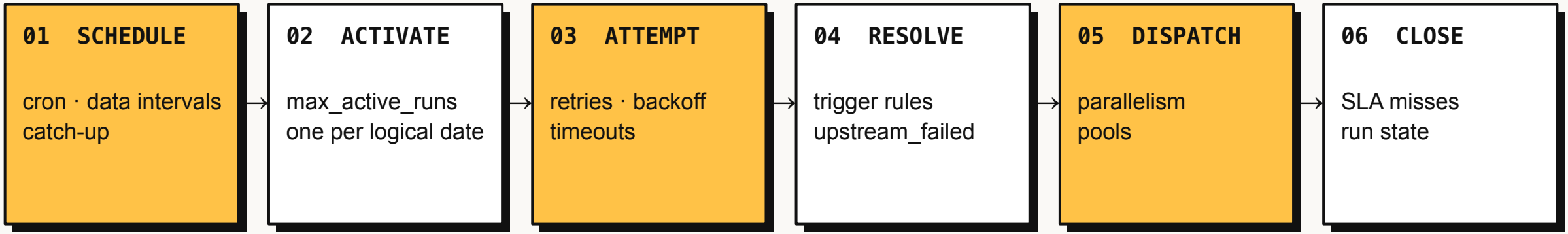
Backfills are risky

Re-running history double-loads data unless every run is idempotent per logical date.

The response: a real scheduler loop you can watch, break and replay, with a doctor that explains it.

03

ONE HEARTBEAT, SIX STEPS



Deterministic by design

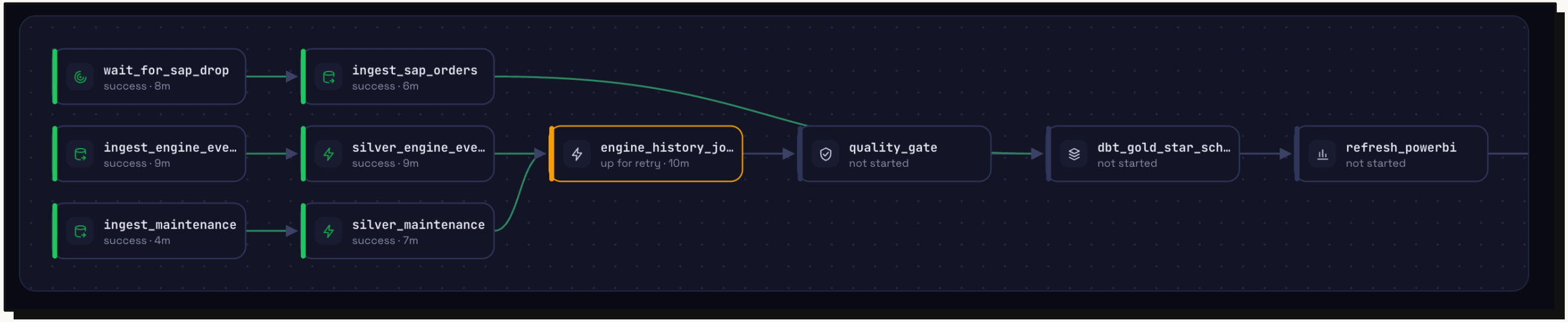
Every outcome is seeded by DAG, logical date, task and try, so a scenario replays exactly and the engine is covered by unit tests: catch-up, backoff, trigger rules, timeouts, pools, backfill.

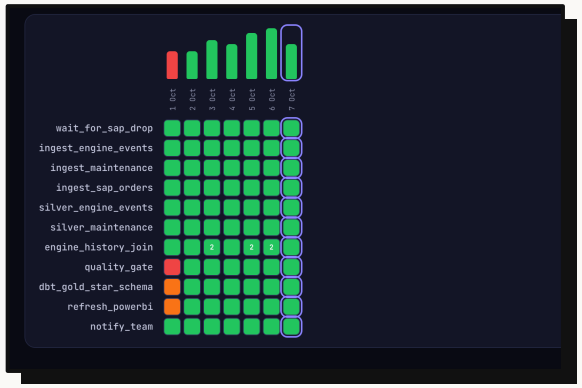
DAG-as-code

DAGs are YAML, validated like a scheduler would at import (cycles, unknown upstreams, bad cron, undeclared pools) and exported as an Apache Airflow 2 file.

04

WATCH IT RUN, THEN READ THE PATTERN





Live graph

Tasks take the state of the selected run; edges into running tasks animate.

Airflow-style grid

One column per run: the red day, the blocked tasks and the retry counts stand out at once.

05

A FLAKY JOIN, RETRIED WITH BACKOFF

engine_history_join

lakehouse_daily · 2026-10-03 · spark

success tries 2 / 3 timeout 1h rule all_success

Joins engine events to maintenance on engine_id (a few engines are very hot)

Clear + downstream | Mark success | Inject failure

Try 1 failed · 9m

Try 2 success · 19m

executor out of memory (skewed join)

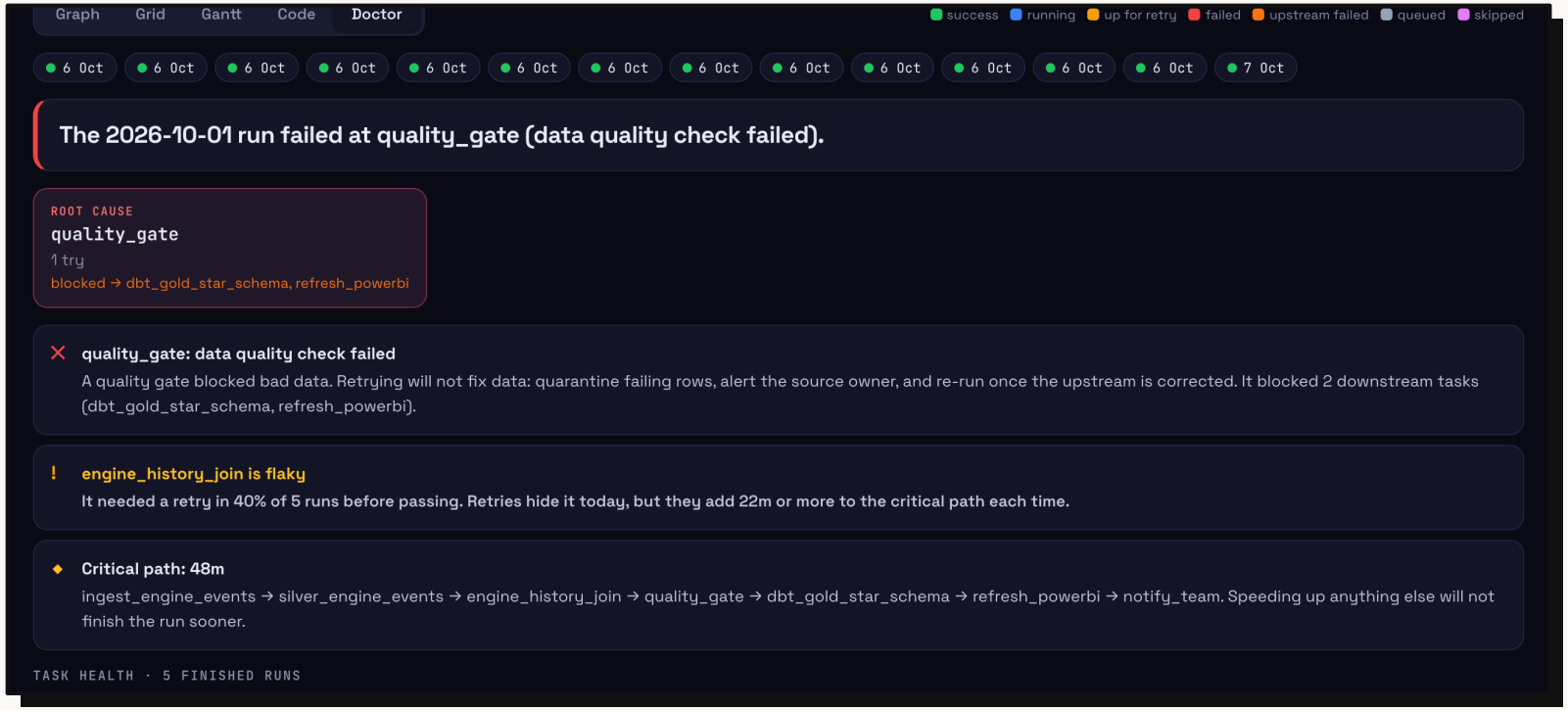
```
[2026-10-06 02:55] INFO  Starting attempt 1 of
engine_history_join (spark) for logical date 2026-10-03
[2026-10-06 02:55] INFO  Submitting job to cluster de-prod-01 (8
× Standard_E8ds_v5, Photon)
[2026-10-06 02:56] INFO  AQE enabled · shuffle partitions 400 ·
input 23 GB
[2026-10-06 03:04] WARN  Stage 14: 399/400 tasks finished in 41s;
task 212 still running (input 18.4 GB vs median 46 MB)
[2026-10-06 03:04] ERROR java.lang.OutOfMemoryError: Java heap
space — executor 7 lost while processing hot key engine_id=ENG-
0042
[2026-10-06 03:04] ERROR Task failed
[2026-10-06 03:04] INFO  Marking task as UP_FOR_RETRY; retry 1 of
```



Try 1 dies on a skewed key (OOM), waits 5 min, try 2 passes. The Gantt shows the lost time on the critical path.

06

CAUSE VS BLAST RADIUS, IN PLAIN ENGLISH



Root cause

quality_gate failed on bad data and blocked dbt and Power BI downstream.

Transient or deterministic

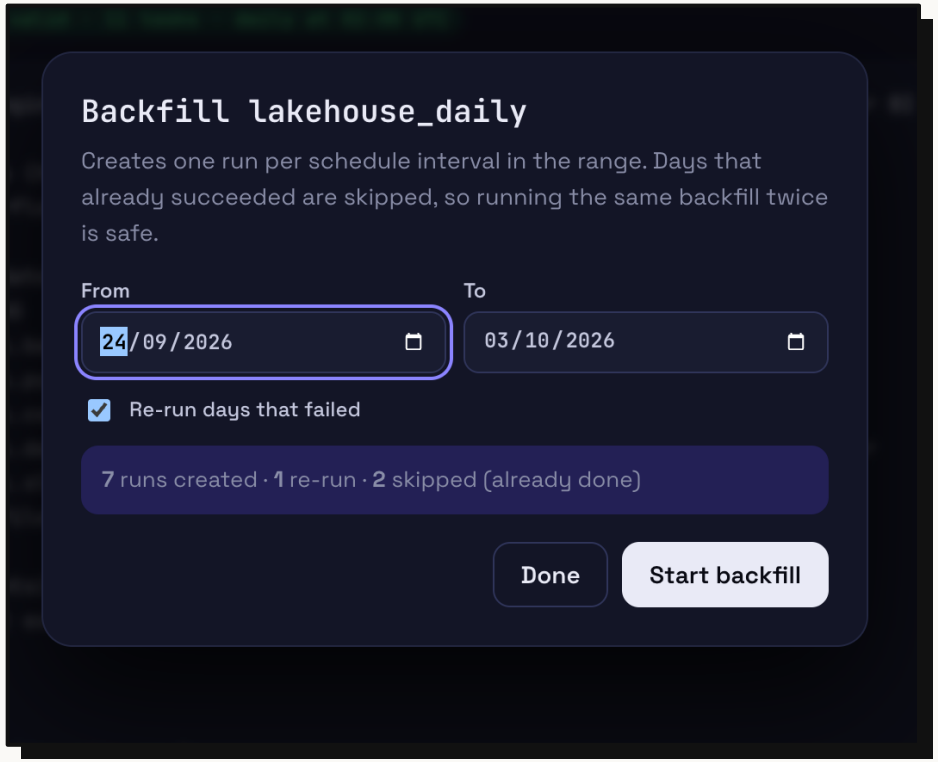
Retries are advised for connection errors, never for a failing quality check.

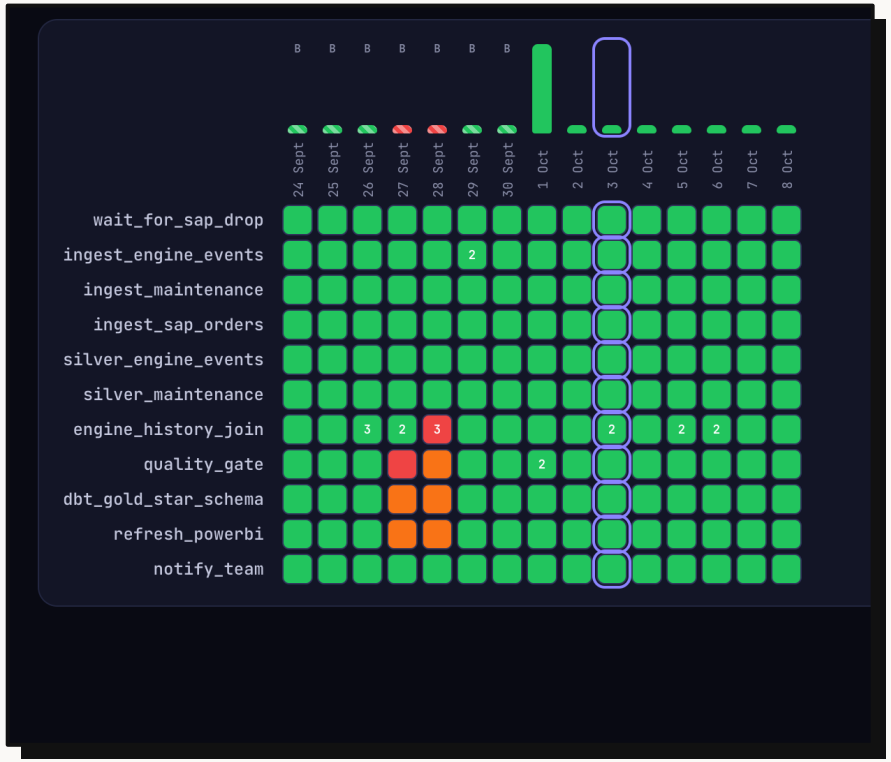
Flaky and slow tasks

Retry-to-pass rate, p95 vs timeout, and the critical path across runs.

07

BACKFILLS THAT ARE SAFE TO REPEAT

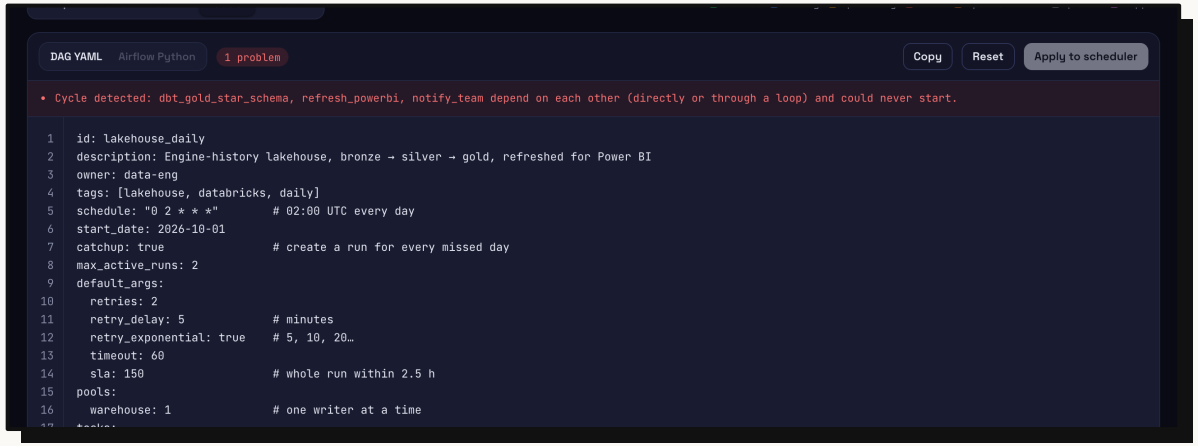


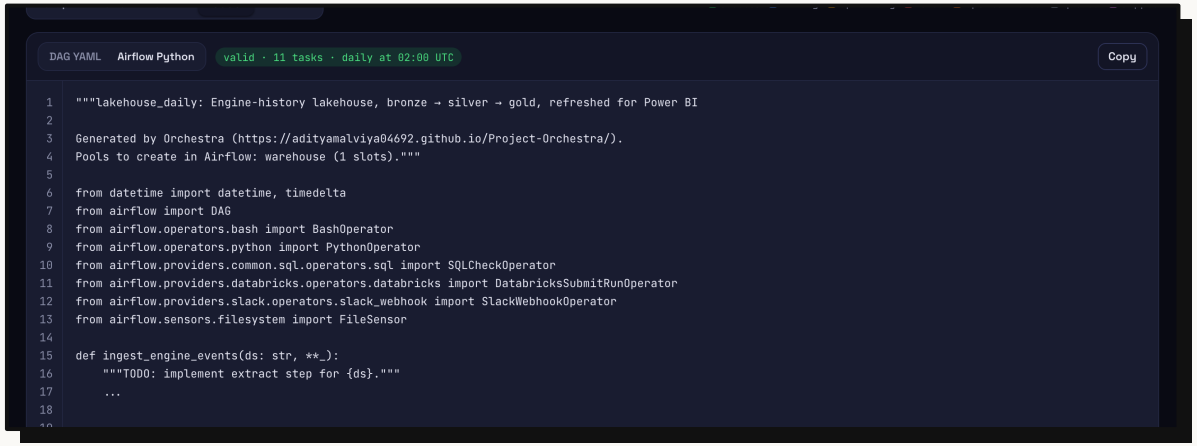


Same range again: 0 created, 10 skipped. One run per logical date; failed days re-run only on request.

08

DAG-AS-CODE, READY FOR AIRFLOW





A cycle is caught before it reaches the scheduler

Retries, backoff, SLAs, pools and trigger rules carry over

adityamalviya04692.github.io/Project-Orchestra/

github.com/adityamalviya04692/Project-Orchestra